MODELAGEM GEOMÉTRICA · PROF. JOAQUIM BENTO

Modelador CSG

Primitivas implementadas: Bloco, Esfera, Cilindro e Cone;

Icaro Mayan Lima Pereira



Roteiro

01

Primitivas

02

CSG

03

SMC

04

RayCast

05

I/O

06

Tema

PARTE 01

Primitivas

Para essa implementação, todas as primitivas são parametrizadas com tamanho 1 e geradas ao redor da origem, ou seja, o centro de todas elas no momento de sua geração é (0, 0, 0).

PRIMITIVAS

Classe abstrata que engloba todas as outras e guarda informações comuns a todas as primitivas

- IntersectRay(): Recebe um raio e retorna as interseções entre esse raio e a primitiva;
- CheckIsInside(): Recebe um ponto e retorna se está dentro, fora ou na fronteira da primitiva;
- CheckIsCellInside(): Recebe uma célula e retorna se está dentro ou fora da primitiva;
- GetCSGBoundingBox(): Retorna uma célula que engloba toda a primitiva;

> **OBSERVAÇÃO**
>
> Para todas as primitivas, as funções CheckIsCellInside e GetCSGBoundingBox são triviais, já que foram implementadas na octree, a única diferença é elas não consideram transformações.

PRIMITIVAS · ESFERA

Interseção ponto-esfera

Como as primitivas são parametrizadas e centradas na origem, para verificar se um ponto está dentro da esfera basta verificar se a distância do ponto até a origem é menor ou igual ao raio da esfera:

$d = \sqrt{(x^2 + y^2 + z^2)}$, se d < raio retorna IN; se d = raio retorna ON; se não retorna OUT

PRIMITIVAS · ESFERA

Interseção raio-esfera

> **OBSERVAÇÃO**
> Um raio é uma semi-reta definida como: um ponto de origem e um vetor de direção.

Para encontrar as interseções de uma reta e uma esfera, vamos resolver a equação da esfera considerando a semi-reta do raio:

Equação da esfera: $x^2 + y^2 + z^2 = r^2$

Equação do raio: P(t) = O + t*D, onde O é a origem e D é a distância, dividindo essa equação para cada eixo temos:

x = O.x + t*D.x
y = O.y + t*D.y
z = O.z + t*D.z

PRIMITIVAS · ESFERA

Interseção raio-esfera

Substituindo as variáveis na equação da esfera:
$(O.x + t*D.x)^2 + (O.y + t*D.y)^2 + (O.z + t*D.z)^2 = r^2$, desenvolvendo cada termo temos:

$$(O.x + t*D.x)^2 = O.x^2 + 2*(O.x*t*D.x) + t^2*D.x^2$$

$$(O.y + t*D.y)^2 = O.y^2 + 2*(O.y*t*D.y) + t^2*D.y^2$$

$$(O.z + t*D.z)^2 = O.z^2 + 2*(O.z*t*D.z) + t^2*D.z^2$$

PRIMITIVAS · ESFERA

Interseção raio-esfera

Agora pando os termos em relação a t e levando tudo para o mesmo lado da equação chegamos a uma equação de segundo grau: $(D.x^2 + D.y^2 + D.z^2)t^2 + 2(O.xD.x + O.yD.y + O.zD.z)t + (O.x^2 + O.y^2 + O.z^2 - r^2) = 0$, ou seja:

$$a = D.x^2 + D.y^2 + D.z^2 \quad ou \quad a = ProdEscalar(D,D)$$

$$b = 2(O.x*D.x + O.y*D.y + O.z*D.z) \quad ou \quad b = 2*ProdEscalar(O,D)$$

$$c = O.x^2 + O.y^2 + O.z^2 - r^2 \quad ou \quad c = ProdEscalar(O,O) - r^2$$

PRIMITIVAS · ESFERA

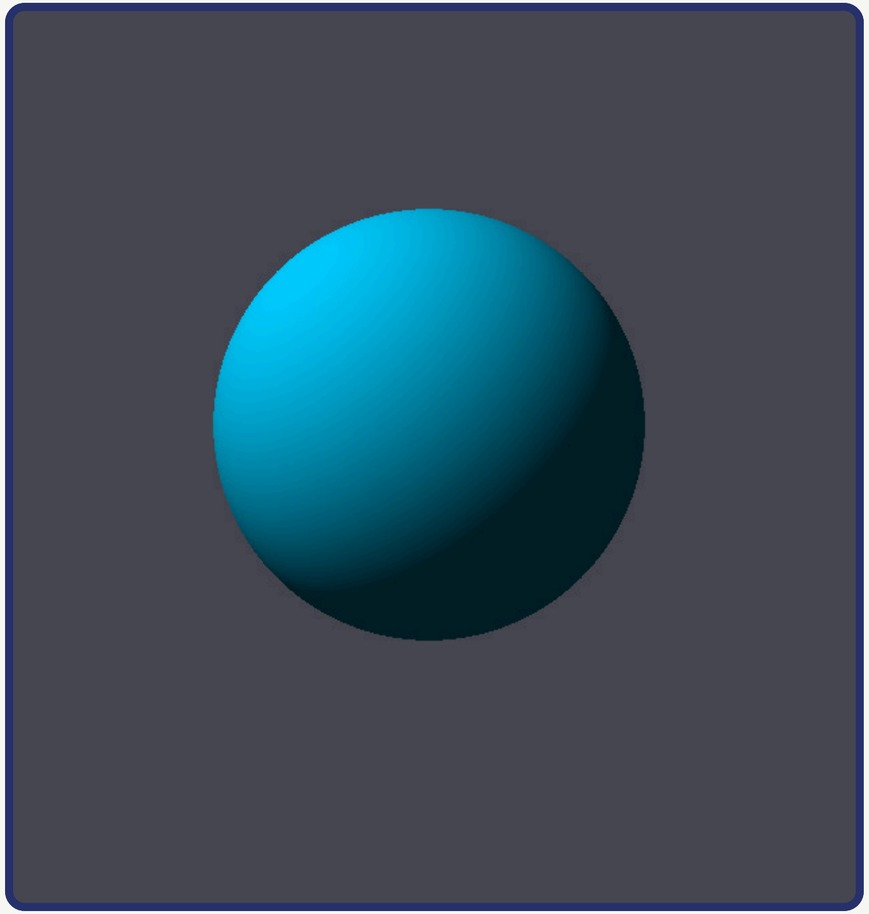
Interseção raio-esfera

Em seguida, basta calcular o delta e verificar:

Se Δ < 0 a equação não tem raízes reais;
Se Δ = 0 a equação tem 1 raíz (Intersecta em 1 ponto);
Se Δ > 0 a equação tem 2 raíz (Intersecta em 2 pontos);

Se há interseções, precisamos verificar se a raiz é positiva (ou seja, está a frente da origem), caso esteja, podemos obter o ponto aplicando na equação do raio P(t) = O + t*D

Ao final, retornamos um vetor com as interseções.



PRIMITIVAS · BLOCO

Interseção ponto-bloco

Para verificar se um ponto está dentro de um bloco centrado na origem, basta verificar individualmente para cada dimensão do ponto se:

Se |x| < tamanho/2 e |y| < tamanho/2 e |z| < tamanho/2 O ponto está IN

Se |x| > tamanho/2 ou |y| > tamanho/2 ou |z| > tamanho/2 O ponto está OUT

Se não, o ponto está ON

PRIMITIVAS · BLOCO

Interseção raio-bloco

Para encontrar as interseções de uma reta e um bloco, vamos utilizar os limites do bloco em cada eixo: -tamanho/2 ≤ x,y,z ≤ tamanho/2 e a equação do raio: P(t) = O + t*D.
Dividindo novamente a equação para cada eixo temos:

$$x = O.x + t*D.x \quad \text{ou seja} \quad -tamanho/2 \leq O.x + t*D.x \leq tamanho/2$$

$$y = O.y + t*D.y \quad \text{ou seja} \quad -tamanho/2 \leq O.y + t*D.y \leq tamanho/2$$

$$z = O.z + t*D.z \quad \text{ou seja} \quad -tamanho/2 \leq O.z + t*D.z \leq tamanho/2$$

PRIMITIVAS · BLOCO

Interseção raio-bloco

Isolando t em cada eixo, vamos calcular a interseção do raio com o plano:

$$\text{Em } X: \quad t1x = (-tamanho.x - O.x) / D.x \quad e \quad t2x = (tamanho.x - O.x) / D.x$$

$$\text{Em } Y: \quad t1y = (-tamanho.y - O.y) / D.y \quad e \quad t2y = (tamanho.y - O.y) / D.y$$

$$\text{Em } Z: \quad t1z = (-tamanho.z - O.z) / D.z \quad e \quad t2z = (tamanho.z - O.z) / D.z$$

PRIMITIVAS · BLOCO

Interseção raio-bloco

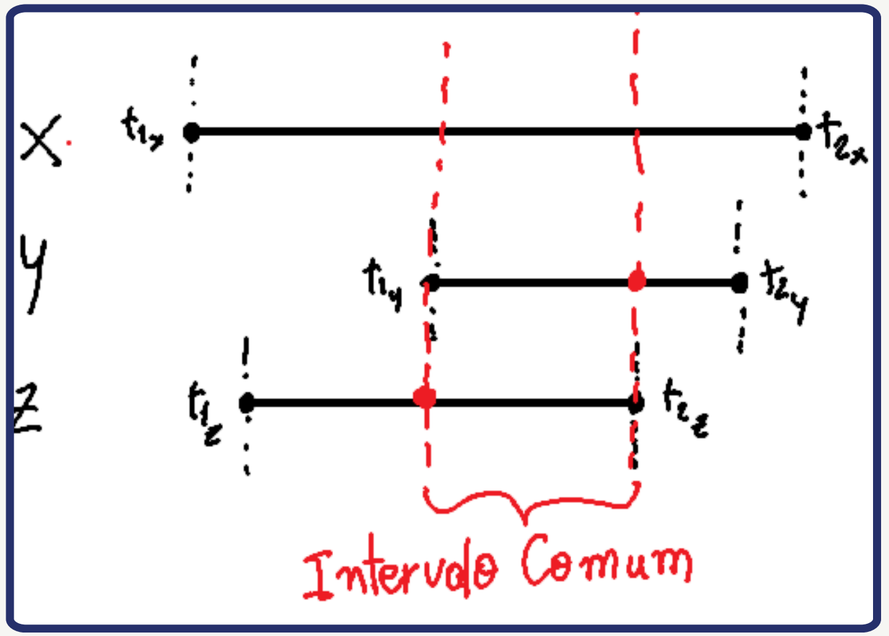
Porém, obter a interseção com um dos planos não garante que o raio esteja atravessando o bloco, então devemos considerar os 3 planos simultaneamente, então a partir desses intervalos calculados para cada plano escolhemos o intervalo comum a todos os eixos

Se os intervalos não possuem uma região em comum, portanto o raio não intersecta o bloco

Se existe um único ponto de contato o raio é tangente ao bloco

Se existe um intervalo dentro do cubo o raio atravessa o bloco

Por fim, para saber os pontos de interseção, basta aplicarmos novamente o intervalo comum a equação do raio P(t) = O + t*D.



PRIMITIVAS · BLOCO

Interseção raio-bloco

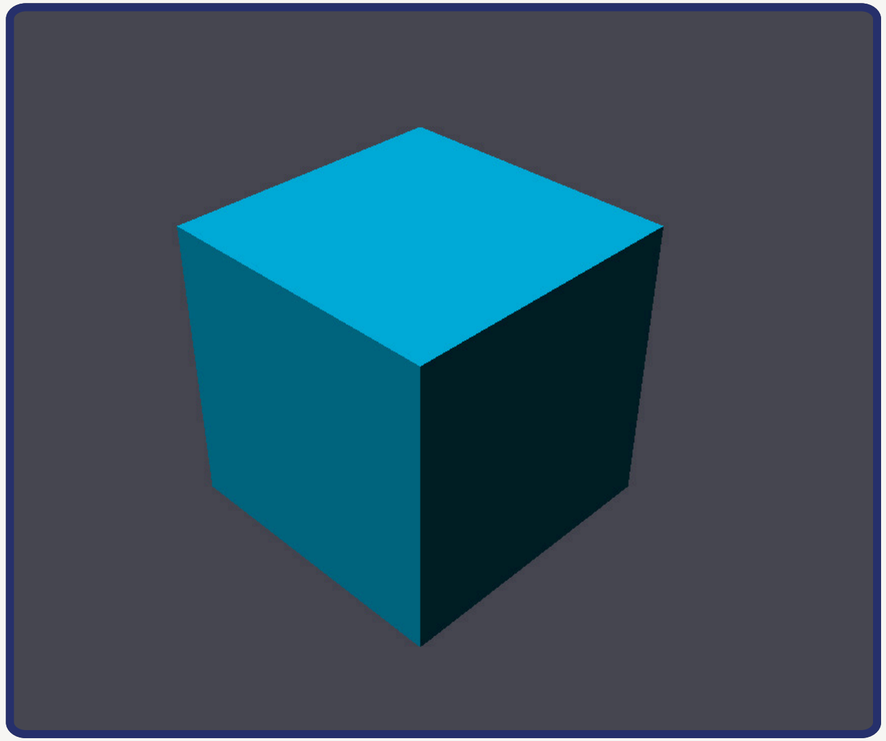
Porém, obter a interseção com um dos planos não garante que o raio esteja atravessando o bloco, então devemos considerar os 3 planos simultaneamente, então a partir desses intervalos calculados para cada plano escolhemos o intervalo comum a todos os eixos

Se os intervalos não possuem uma região em comum, portanto o raio não intersecta o bloco

Se existe um único ponto de contato o raio é tangente ao bloco

Se existe um intervalo dentro do cubo o raio atravessa o bloco

Por fim, para saber os pontos de interseção, basta aplicarmos novamente o intervalo comum a equação do raio $P(t) = O + t*D$.



PRIMITIVAS · CILINDRO

Interseção ponto-cilindro

Para verificar se um ponto está dentro do cilindro é bem semelhante a esfera, vamos calcular a distância do ponto até a origem, desconsiderando o eixo Y $d = \sqrt{(x^2 + z^2)}$

Se d < raio e |y| < altura/2 o ponto está IN

Se d <= raio e |y| = altura/2 ou d = raio e |y| ≤ altura/2 o ponto está ON

Se não o ponto está OUT

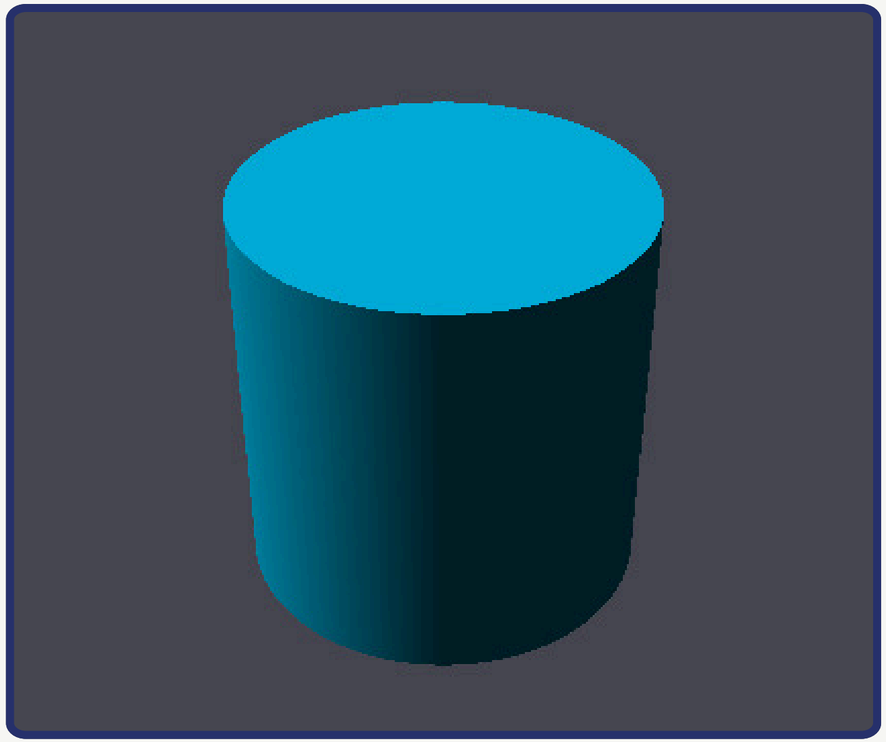
PRIMITIVAS · CILINDRO

Interseção raio-cilindro

Para encontrar as interseções de uma raio e um cilindro, dividimos em duas etapas das validações que já vimos anteriormente:

Aplicamos a mesma equação da esfera desconsiderando o eixo Y para verificar se há interseção na superfície lateral, se sim, verificamos se essa intersecção respeita os limites de altura do cilindro.

Em seguida, verificamos as interseções com os dois planos (altura/2 e -altura/2), usando o mesmo método do bloco, mas para cada ponto encontrado, verificamos se ele está dentro do limite do raio, usando a mesma verificação da intersecção ponto-cilindro ($d = \sqrt{(x^2 + z^2)}$).

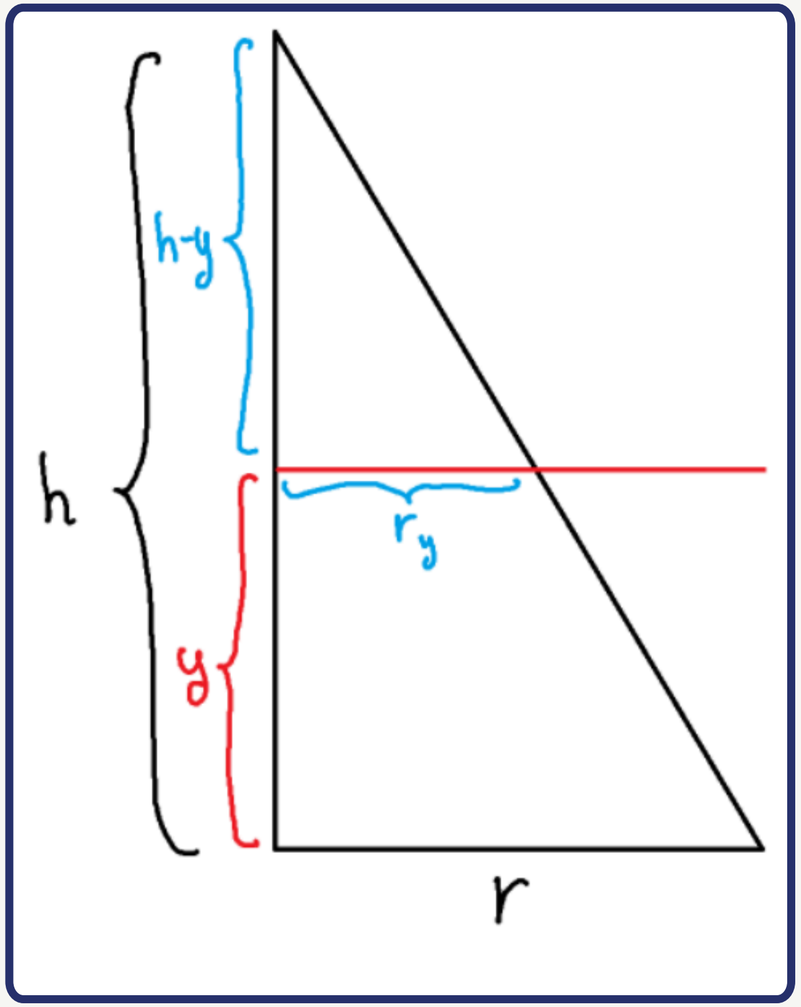


PRIMITIVAS · CONE

Interseção ponto-cone

Para verificar se um ponto está dentro do cone, dividimos a análise em duas etapas:

Primeiro calculamos o raio do cone na altura y do ponto utilizando semelhança de triângulos:



PRIMITIVAS · CONE

Interseção ponto-cone

Ou seja, o raio em y é $h/r = (h - y)/ry \Rightarrow ry*h = r(h - y) \Rightarrow ry = r(h - y)/h$, e a distância $d = \sqrt{(x^2 + z^2)}$

Se y estiver fora dos limites de altura do cone o ponto está OUT

Se d = ry e y estiver nos limites de altura o ponto está ON

Se d < ry e y estiver dentro nos limites de altura o ponto está IN

Se não, o ponto está OUT

PRIMITIVAS · CONE

Interseção raio-cone

Para encontrar as interseções de uma reta e um cone, vamos resolver a equação da superfície lateral do cone considerando a semi-reta do raio:
Equação da superfície lateral: $x^2 + z^2 = (r/h)^2(h/2 - y)^2$
Equação do raio: P(t) = O + t*D, dividindo essa equação para cada eixo temos:

$$x = O.x + t*D.x$$

$$y = O.y + t*D.y$$

$$z = O.z + t*D.z$$

PRIMITIVAS · CONE

Interseção raio-cone

Substituindo as variáveis na equação:
$(O.x + t*D.x)^2 + (O.z + t*D.z)^2 = (r/h)^2(h/2 - (O.y - t*D.y))^2$
Expandindo e agrupando os termos em função de t obtemos uma equação quadrática

> **OBSERVAÇÃO**
>
> Aqui aplicamos o mesmo processo de expandir e agrupar os termos que aplicamos na equação da esfera, por isso foi ocultado.

$$(D.x^2 + D.z^2 - D.y^2)t^2 + 2(O.xD.x + O.zD.z + (r/h)^2(h/2 - O.y)D.y)t + O.x^2 + O.z^2 - (r/h)^2(h/2 - O.y)^2$$

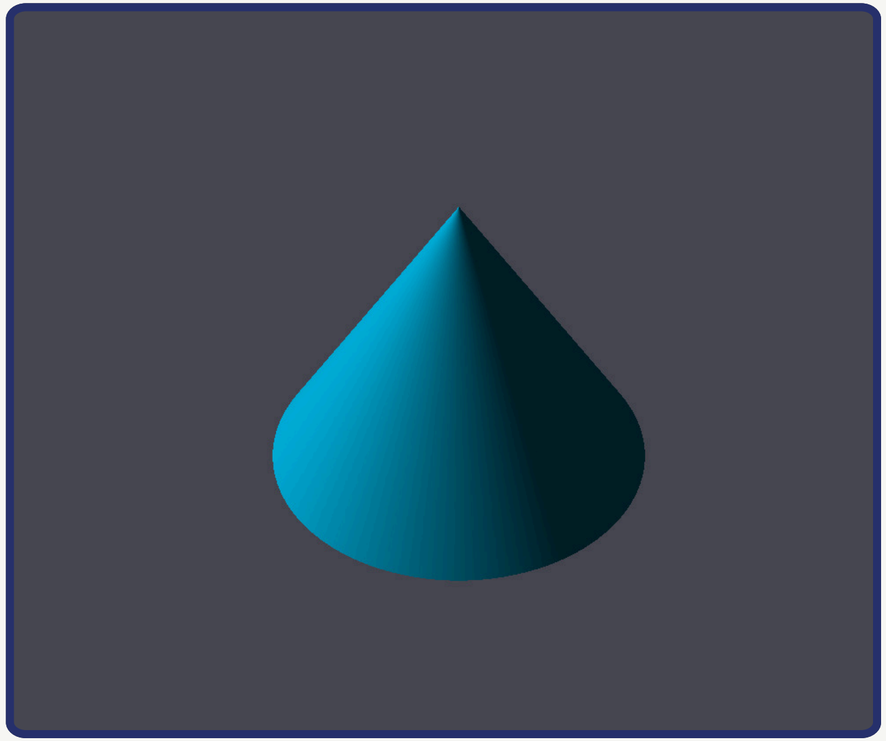
Interseção raio-cone

Em seguida, obtemos os valores de a, b e c e resolvemos a equação:

Se Δ < 0 não existe raiz real;
Se Δ = 0 existe raiz;
Se Δ > 0 existem duas raízes;

Caso exista, calculamos as raízes e podemos obter o ponto aplicando novamente na equação do raio P(t) = O + t*D, também verificamos se y dos pontos de interseção estão nos limites de altura do cone para garantir que a interseção é na superfície finita do cone.

Por fim, usamos a mesma verificação das superfícies planas do cilindro para a superfície inferior do cone e verificamos se a interseção no plano tem a distância menor que o raio, ao final, agrupamos todas as interseções e retornamos.



PARTE 02

CSG

CSG · NÓS

Todo nó tem uma referência para seus filhos esquerdo e direito, um tipo de nó e um valor.

Primitiva

Nós do tipo primitiva sempre são folhas (por consequência não tem filhos) e seu valor é uma das primitivas citadas anteriormente.

Transformação

Nós do tipo transformação aponta para apenas um filho, no qual será aplicada a transformação, um tipo de transformação (escala, rotação ou translação) e um vetor de tamanho 3 que representa o valor da transformação em cada eixo.

Booleano

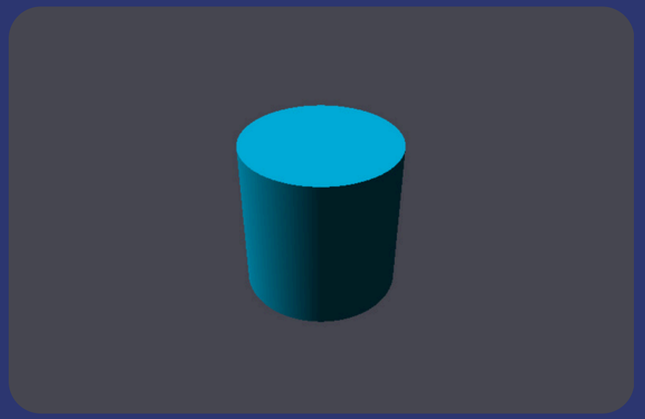
Nós do tipo booleanos sempre apontam para dois filhos, nos quais serão aplicadas a operação (a prioridade sempre é da esquerda para a direita) e um tipo de operação (união, interseção ou diferença).

CSG · NÓS

Todo nó tem uma referência para seus filhos esquerdo e direito, um tipo de nó e um valor.

CreatePrimitiveNode

Recebe uma primitiva e retorna um nó do tipo primitive.



CreateTransformNode

Recebe um tipo de transformação, um vetor float de tamanho 3 e um nó e retorna um nó do tipo transform.

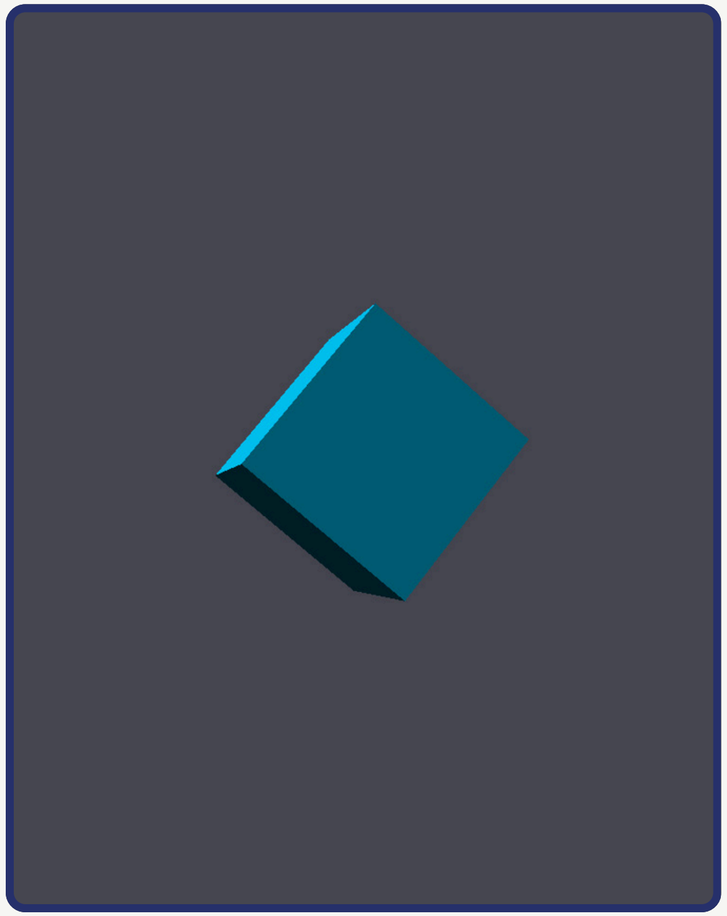


CreateBooleanNode

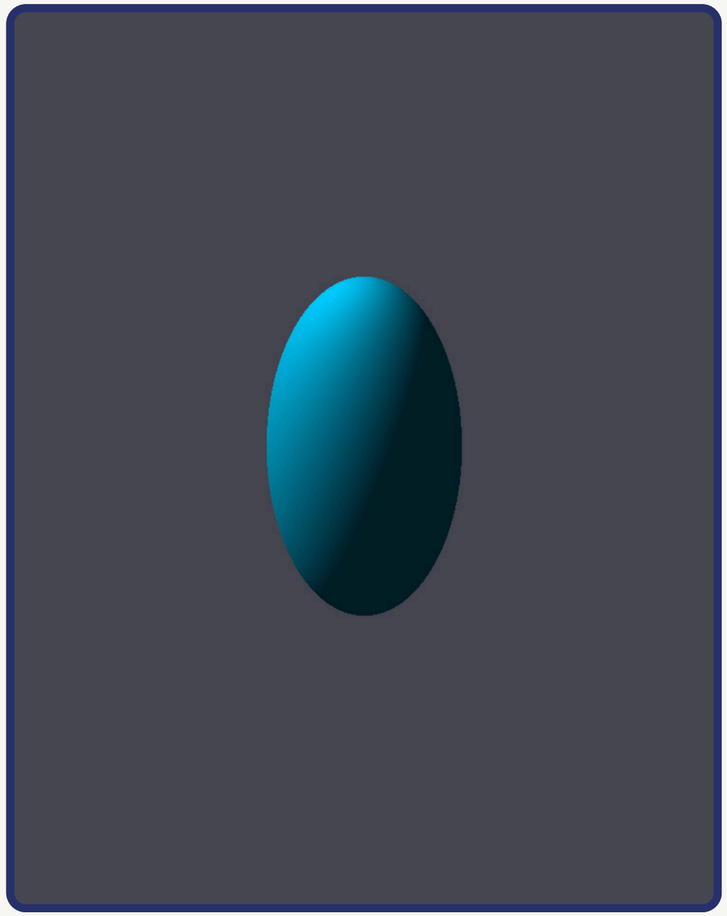
Recebe um tipo de operação e dois nós e retorna um nó do tipo boolean.



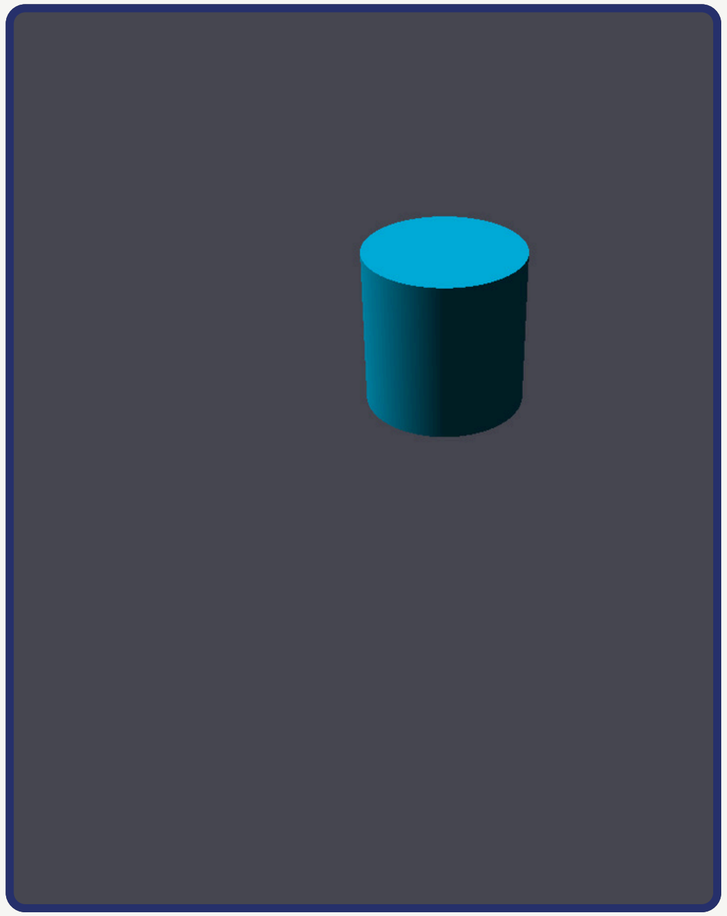
Exemplos



Rotação

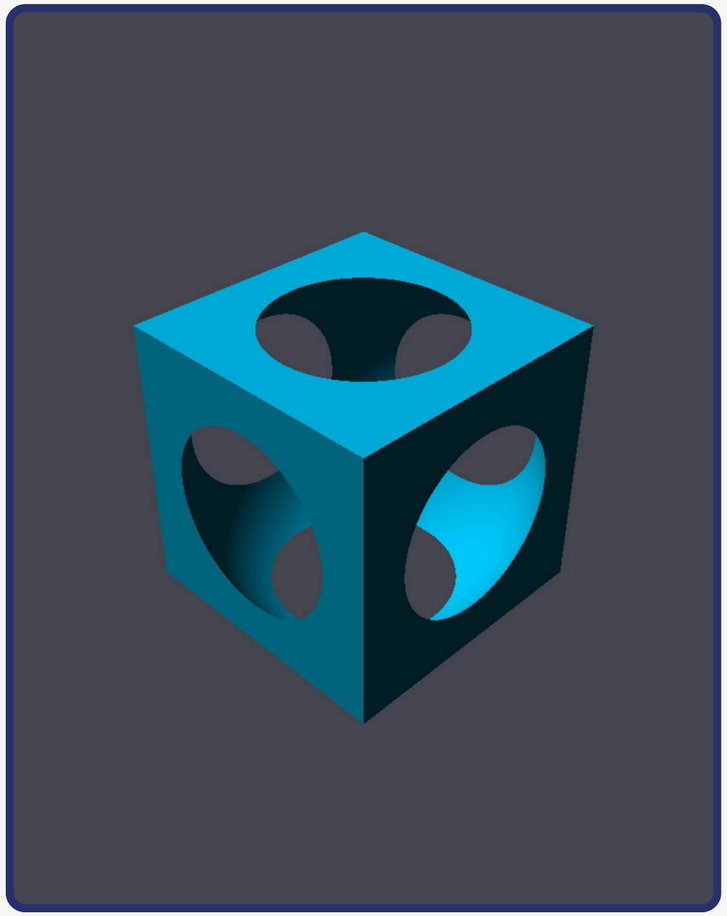


Escala

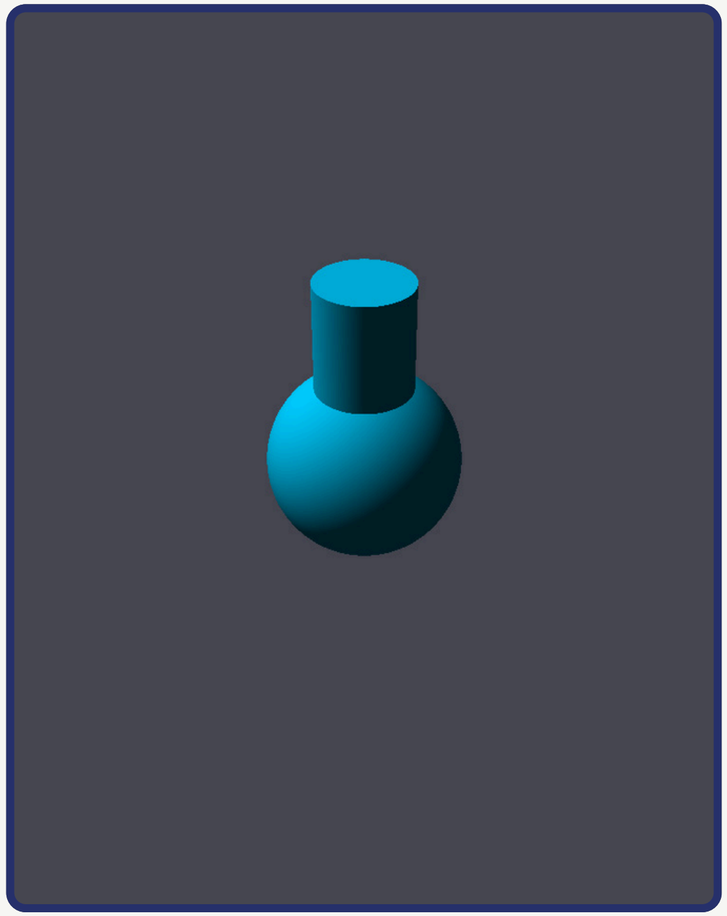


Translação

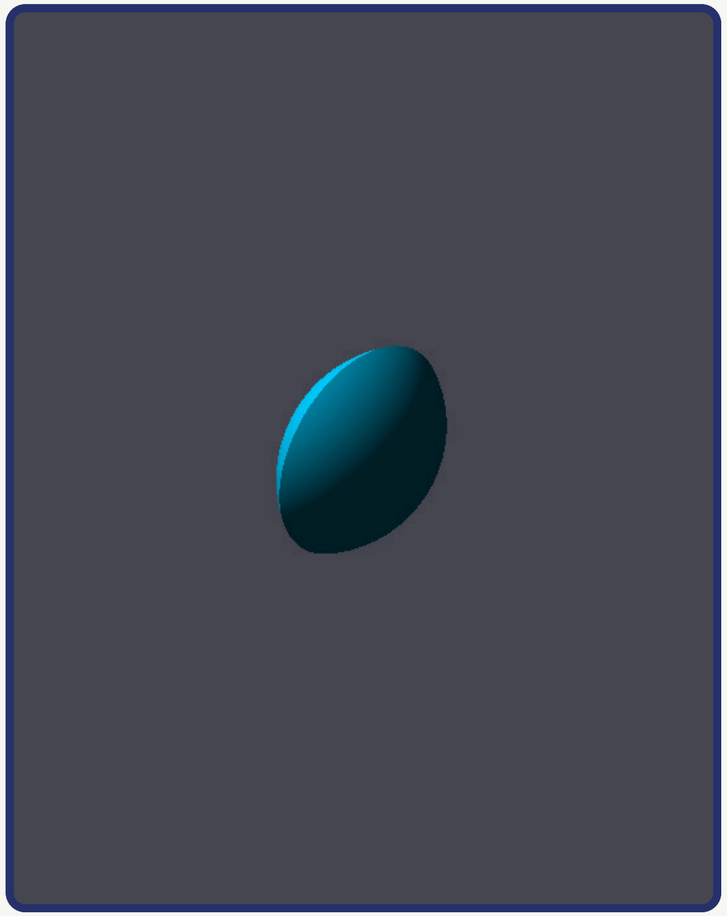
Exemplos



Diferença



União



Intersecção

CSG

Geração

A geração da árvore se dá apenas ao empilhamento de nós em uma árvore binária, só precisamos criar um nó através das funções (CreatePrimitiveNode, CreateBooleanNode e CreateTransformNode) e apontar a raiz da árvore para algum nó (SetRoot), isso faz também com que os operadores geométricos e booleanos se tornem triviais, uma vez que basta criar o nó e a representação da operação já está aplicada.

CSG

Interseção ponto-CSG

Também conseguimos verificar se um ponto está ou não dentro do sólido a partir da árvore csg utilizando combinações das implementações anteriores:

- Se o nó é do tipo primitiva: retorne a classificação ponto-primitiva
- Se o nó é do tipo transformação: aplique a transformação inversa dos valores e retorne recursivamente a classificação ponto-primitiva usando o novo ponto
- Se o nó é do tipo booleano: recursivamente obtenha a classificação de cada um dos seus filhos

Se o nó é do tipo booleano: recursivamente obtenha a classificação de cada um dos seus filhos

Se for do tipo união:

Se em algum dos dois filhos o ponto está ON, retorne ON;

Se em algum dos dois filhos o ponto está IN, retorne IN;

Se não, retorne OUT;

Se for do tipo interseção:

Se em algum dos dois filhos o ponto está OUT, retorne OUT;

Se em algum dos dois filhos o ponto está ON, retorne ON;

Se não retorne IN;

Se for do tipo diferença:

Se o esquerdo retornar OUT ou o direito retornar IN, retorne OUT;

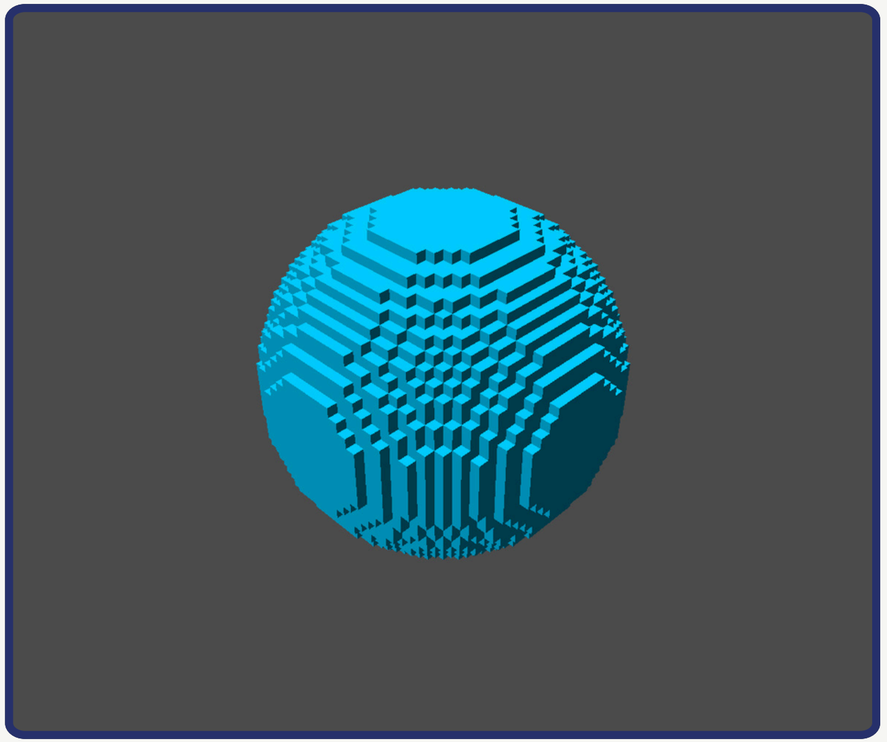
Se em algum dos dois filhos o ponto está ON, retorne ON;

Se não retorne IN;

CSG para Octree

OBSERVAÇÃO

A partir da Octree conseguimos calcular a área, o volume, o wireframe e também visualizar em tempo real no modelador para obter uma responsividade rápida da árvore pré-renderizada.



BoundingBox

Para obter o Bounding Box da árvore CSG basta descer recursivamente na árvore até as folhas, gerar o bounding box da respectiva primitiva, ao subir na árvore precisamos tratar a nossa célula para os nós de transformação e booleanos:

- Se o nó atual for uma transformação: aplicamos a mesma transformação na célula;
    - Com exceção da rotação, pois o bounding box precisa estar alinhado com os eixos XYZ, assim, ao rotacionar a célula, obtemos seus 8 vértices e criamos uma nova bounding box que engloba os 8 vértices rotacionados
- Se o nó atual for uma operação booleana:
    - União: calculamos a menor caixa cúbica que engloba as caixas dos dois filhos, usando as extremidades da célula obtida dos seus pontos mínimos e máximos.
    - Interseção: calcula a região de sobreposição entre as duas caixas. Se não houver sobreposição, retorna uma célula vazia de tamanho zero.
    - Diferença: retorna a caixa do filho esquerdo, pois a subtração de B em A não pode produzir um volume fora dos limites de A.

Interseção célula-CSG

A partir da bounding box, basta realizar o mesmo procedimento da geração da octree, verificamos se a célula está dentro, fora ou intersectando a árvore CSG, pintamos o nó de black, white ou instanciamos seus 8 filhos e repetimos a verificação para todos.

Para verificar se uma célula está ou não dentro da árvore basta descer recursivamente até as primitivas e verificar utilizando a mesma função da Octree, tratando a nossa célula para os nós de transformação e booleanos:

Interseção célula-CSG

- Se o nó atual for uma transformação: aplicamos a transformação inversa nos 8 vértices da célula para levá-la para o espaço local e recursivamente checar para o filho e a nova célula;
- Se o nó atual for um booleano: obtemos a classificação da célula para os dois filhos e em seguida aplicamos a operação
    - União: retorna IN se pelo menos um dos filhos classificar a célula como IN. Retorna OUT se ambos forem OUT. Caso contrário, retorna ON (ou intersecting).
    - Interseção: retorna OUT se pelo menos um dos filhos for OUT. Retorna IN se ambos forem IN. Caso contrário, retorna ON.
    - Diferença: retorna OUT se A for OUT ou B for IN. Retorna IN se A for IN e B for OUT. Caso contrário, retorna ON.

PARTE 03

SMC (Set membership Classification)

Essa classe tem um papel específico de classificar um raio em relação a uma árvore CSG.

NormalizeIntervals

> **OBSERVAÇÃO**
> Um intervalo é definido por um struct que possui início, fim e uma classificação.

Recebe dois vetores de intervalos e normaliza os vetores para cada ponto de interseção.

Exemplo
Dado um intervalo representado por {início, fim, classificação}

Intervalos A = [{1, 2, OUT}, {2, 6, IN}, {6, inf., OUT}]
Intervalos B = [{1, 3, OUT}, {3, 5, IN}, {5, inf., OUT}]

Intervalos A normalizado = [{1, 2, OUT}, {2, 3, IN}, {3, 5, IN}, {5, 6, IN}, {6, inf., OUT}]
Intervalos B normalizado = [{1, 2, OUT}, {2, 3, OUT}, {3, 5, IN}, {5, 6, OUT}, {6, inf., OUT}]

SMC · FUNÇÕES AUXILIARES

MergeIntervals

Recebe um vetor de intervalos e junta os intervalos que possuem a mesma classificação consecutivamente.

Exemplo
Intervalos A = [{1, 2, OUT}, {2, 3, IN}, {3, 5, IN}, {5, 6, IN}, {6, inf., OUT}]
Intervalos A merged = [{1, 2, OUT}, {2, 6, IN}, {6, inf., OUT}]

> **OBSERVAÇÃO**
>
> As duas funções auxiliares são basicamente o inverso uma da outra, pois, para comparar os booleanos precisamos que os intervalos sejam iguais, mas para retornar o resultado, deve ser o menos poluído possível.

Primitiva

A classificação recebe um raio e uma árvore CSG e é dividida para cada tipo de nó

Se o nó a ser classificado for uma primitiva:

- Primeiro, obtemos todos os pontos de interseção entre o raio e a primitiva utilizando a função de interseção raio-primitiva.
- Em seguida, classificamos a origem do raio utilizando a classificação ponto-primitiva.
    - Se a origem for OUT, criamos um intervalo entre a origem e a primeira interseção, classificado como OUT.
    - Se a origem for IN, criamos um intervalo entre a origem e a primeira interseção, classificado como IN.
    - Se a origem for ON e houver pelo menos uma interseção, criamos um intervalo entre a origem e a primeira interseção e classificamos como IN .
    - Se a origem for ON e não houver interseções, teremos apenas o intervalo {origem, inf., OUT}.

SMC · CLASSIFICAÇÃO

Primitiva

- A partir daí, criamos os intervalos entre as interseções consecutivas (ordenadas em t), alternando as classificações entre IN e OUT a cada cruzamento da superfície. Por fim, criamos o último intervalo, que vai da última interseção até o infinito e é classificado como OUT.
- Ao final, retornamos todos os intervalos classificados.

Se o nó a ser classificado for uma transformação, aplicamos a transformada inversa para cada tipo.

Translação

Criamos um novo raio local deslocando a origem - translação, mantemos a direção e recursivamente classificamos o novo raio com o filho da transformação, e por fim, aplicamos novamente a translação em cada ponto dos intervalos.

Escala

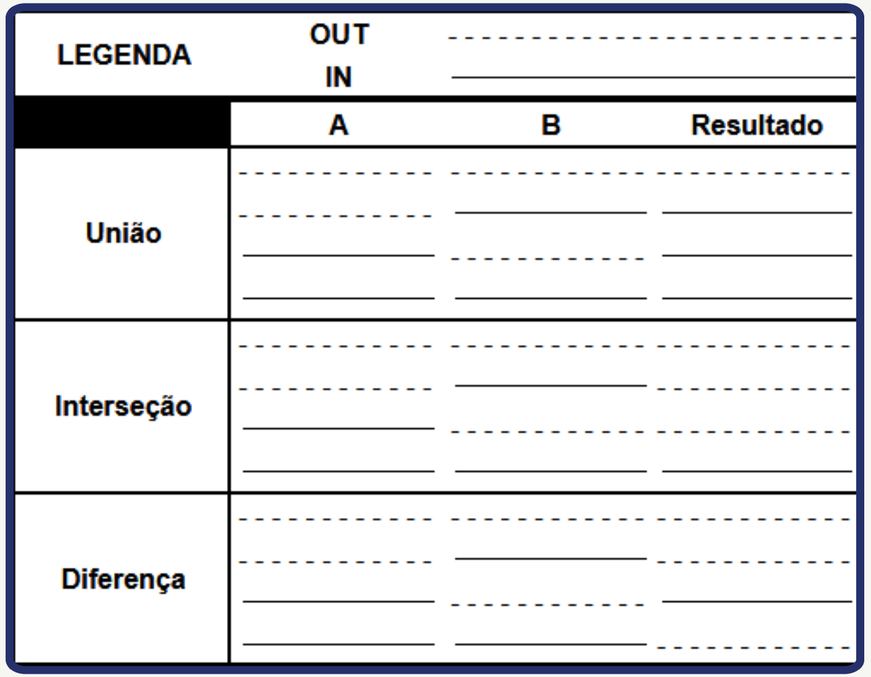
Criamos um novo raio local dividindo a origem e a direção pela escala, recursivamente classificamos o novo raio com o filho da transformação, e por fim, aplicamos novamente a escala em cada ponto dos intervalos.

Rotação

Primeiro criamos a matriz de rotação e obtemos sua inversa por meio da transposta da rotação, em seguida criamos um novo raio local aplicando a matriz inversa na origem e na direção, recursivamente classificamos o novo raio com o filho da transformação, e por fim, aplicamos novamente a rotação em cada ponto dos intervalos.

Booleano

Se o nó a ser classificado for uma operação booleana, primeiro, classificamos recursivamente os dois filhos do nó booleano utilizando o mesmo raio, obtendo os intervalos de cada um. Em seguida, normalizamos os intervalos de A em relação aos de B e vice-versa, garantindo que ambos tenham os mesmos limites e possam ser comparados intervalo a intervalo. Depois, percorremos os intervalos normalizados e aplicamos a operação booleana correspondente.



SMC · CLASSIFICAÇÃO

Booleano

União

Se pelo menos um dos intervalos estiver classificado como IN, o intervalo resultante será IN. Caso contrário, será OUT.

Interseção

Se ambos os intervalos estiverem classificados como IN, o intervalo resultante será IN. Caso contrário, será OUT.

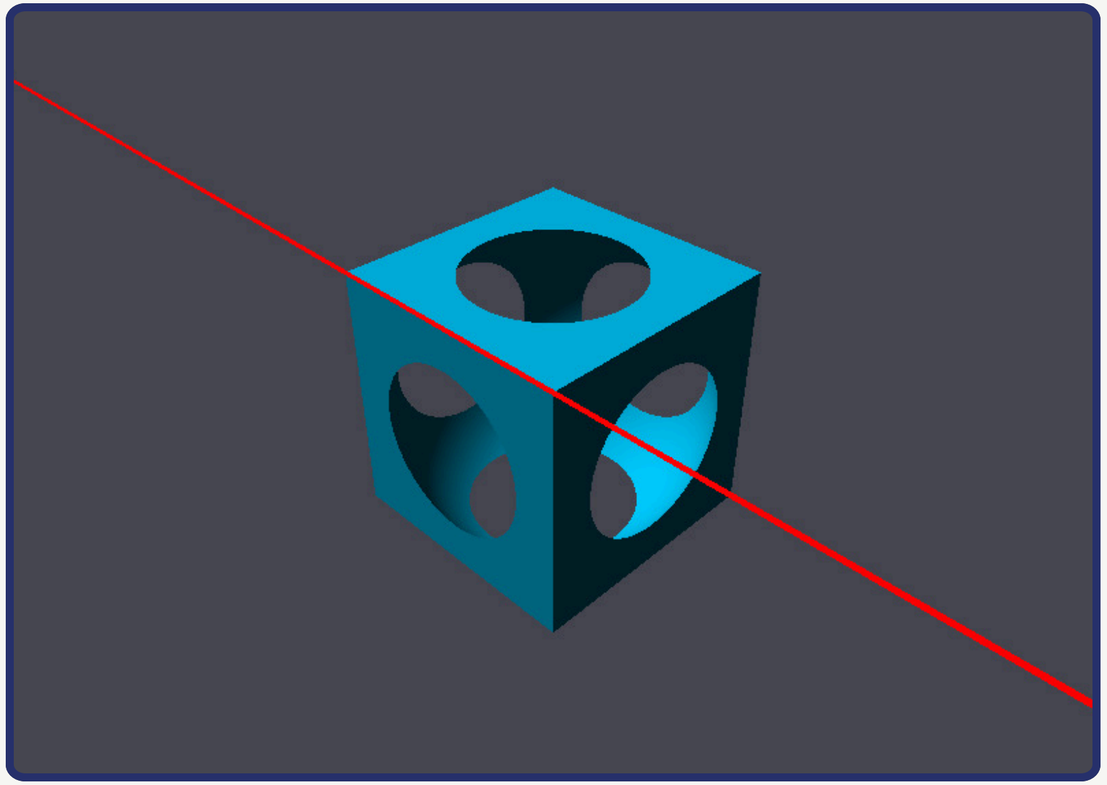
Diferença

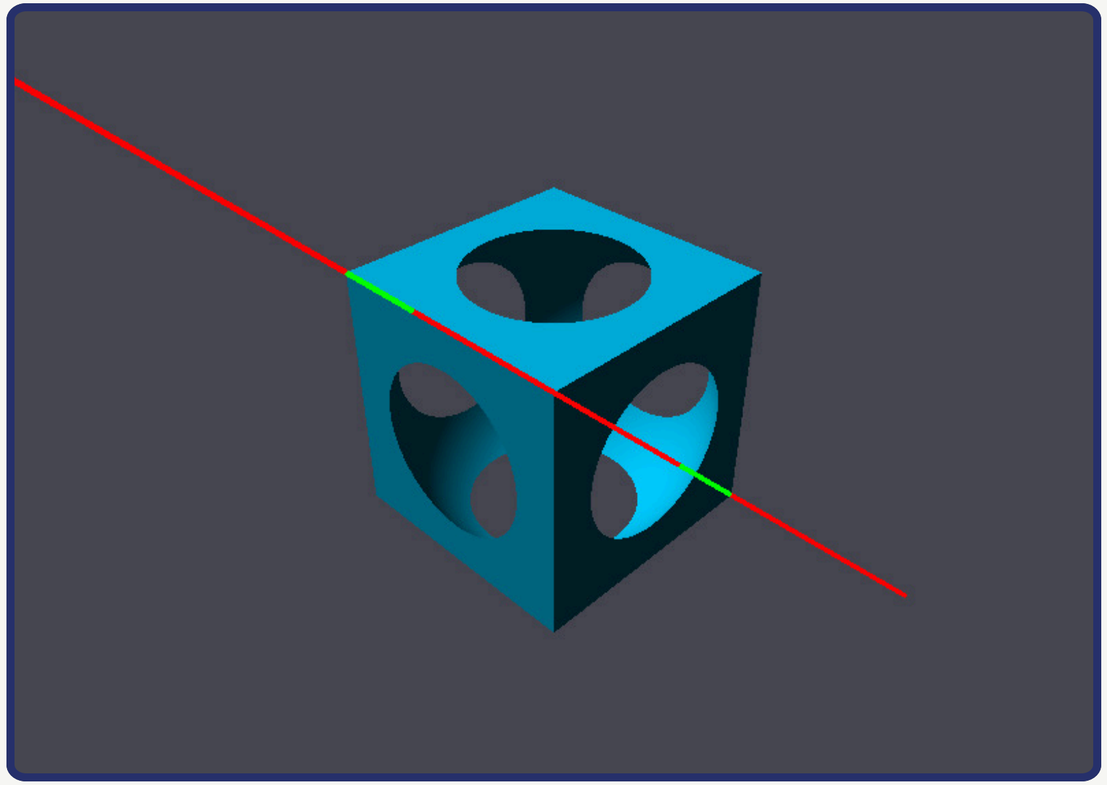
Se o intervalo de A estiver classificado como IN e o de B como OUT, o intervalo resultante será IN. Caso contrário, será OUT.

SMC · CLASSIFICAÇÃO

Booleano

Após classificar todos os intervalos, executamos o Merge Intervals para unir intervalos consecutivos que possuem a mesma classificação, simplificando o resultado





PARTE 04

RayCast

Como não existe malha, para conseguir visualizar a árvore CSG, precisamos implementar o RayCast e renderizar uma imagem utilizando a classificação raio-CSG com o SMC.

RAYCAST

Render

Obtemos a matriz inversa da câmera, com isso, conseguimos recuperar a posição e a orientação da câmera no mundo, permitindo obter os vetores forward, right e up, que representam suas direções frontal, lateral e vertical. Em seguida, calculamos a proporção da renderização (width/height) e utilizamos o FOV de 45° para determinar a abertura dos raios.

Para cada píxel em 0 ≤ x < width e 0 ≤ y < height, convertendo suas coordenadas para o intervalo normalizado de -1 a 1, deslocando a direção central horizontalmente e verticalmente conforme a posição do pixel:

$$posX = (2*x/width) - 1 \quad e \quad posY = 1 - (2*y/height)$$

$$D = F + R*(posX*width/height*0,4142) + U*(posY*0,4142)$$

RAYCAST

Render

Assim, podemos instanciar o nosso raio atual tendo sua origem na mesma origem da câmera e a direção D utilizando a classificação do SMC obtemos os intervalos entre a árvore CSG e nosso raio atual, por fim, pegamos o primeiro hit com a árvore CSG (se existir) calculamos a luz difusa como o produto escalar entre a normal na primeira interseção e a direção da luz (-1, 2, 1):

diffuse = prodEscalar(normal, {-1, 2, 1})

RAYCAST

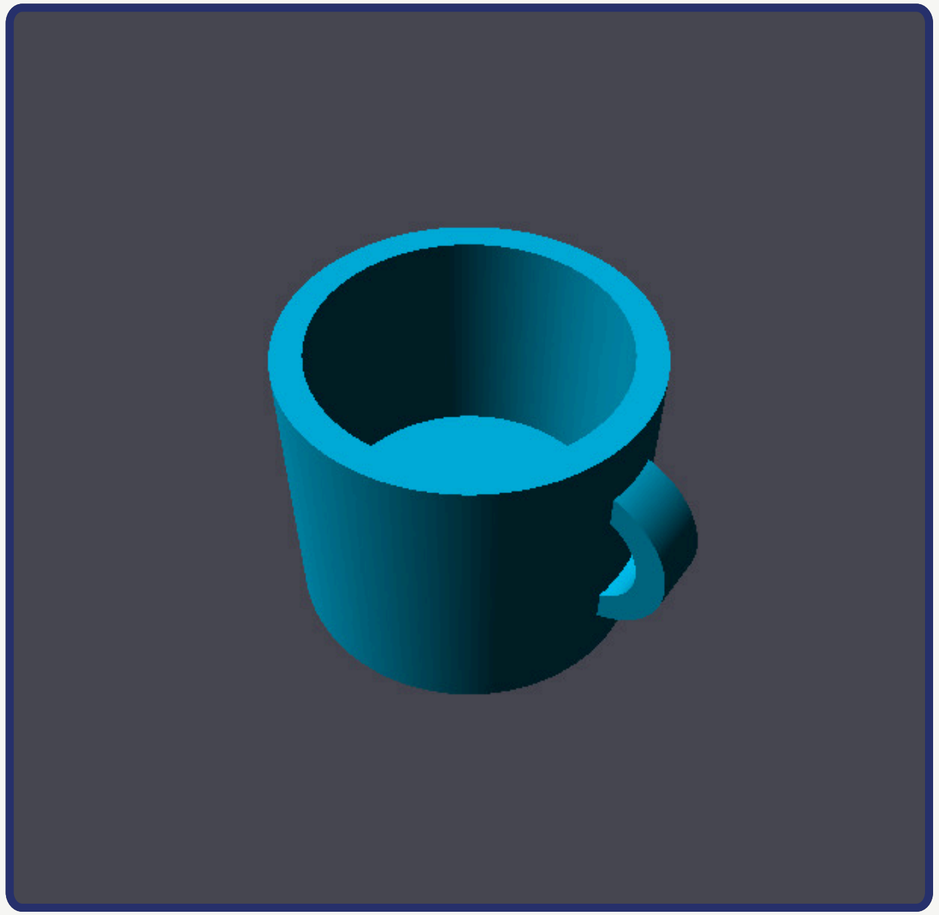
Render

Em seguida calculamos a intensidade da cor nesse pixel combinando uma iluminação ambiente com a luz difusa para evitar pontos totalmente pretos:

intensity = 0,15 + (0,85 * diffuse)

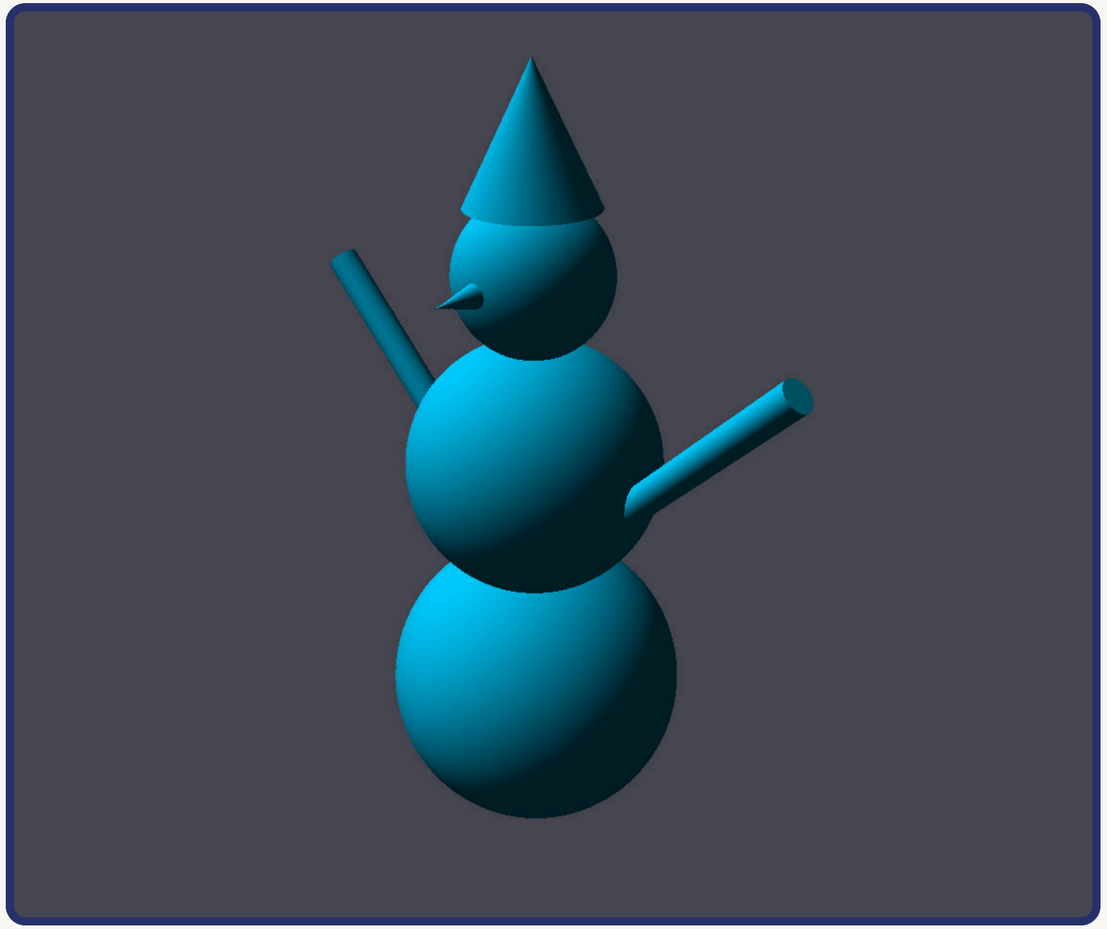
Por fim, pintamos o pixel XY atual com a cor base (fixada em azul) * intensity

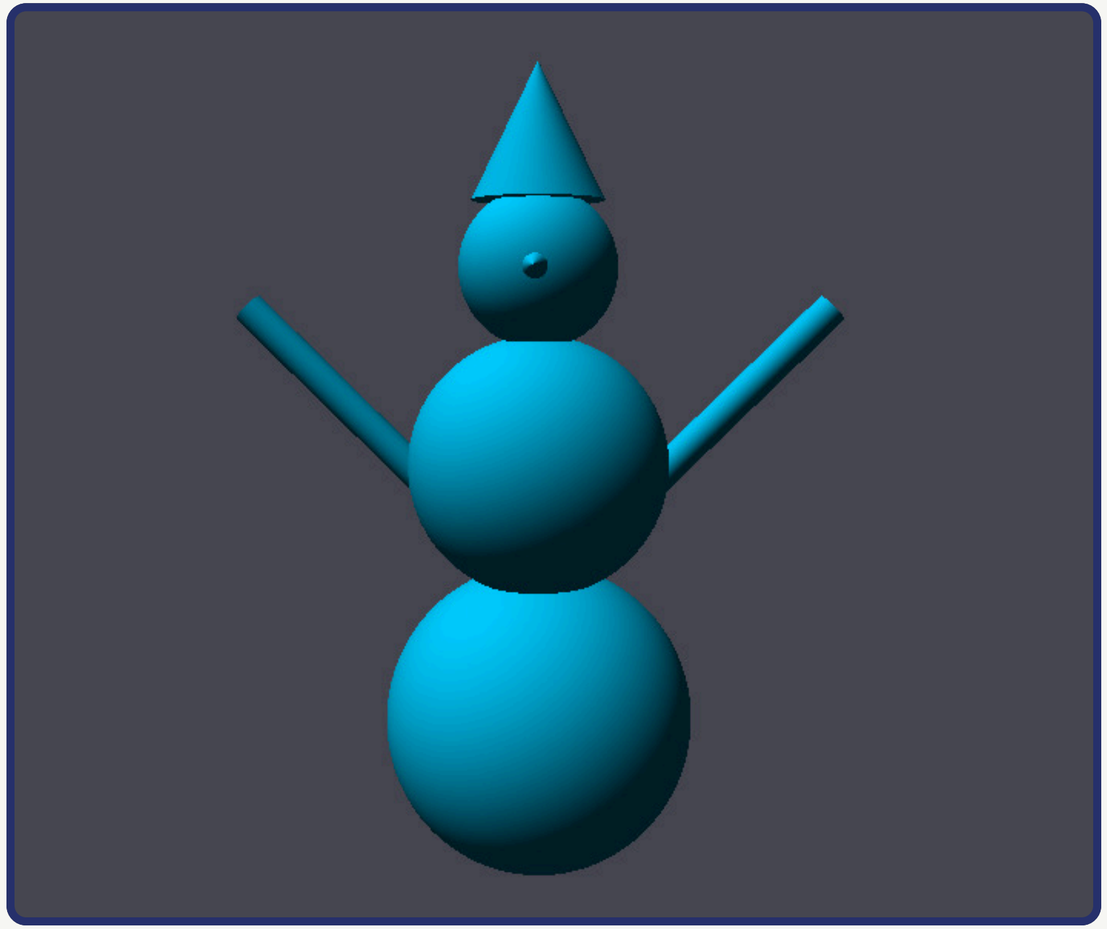
> **OBSERVAÇÃO**
> A normal é calculada previamente durante a interseção raio-CSG para evitar calcular novamente agora.



RAYCAST

Render





PARTE 05

I/O

I/O

Output

A exportação é trivial, basta percorrer a árvore binária utilizando busca em profundidade na pós-ordem, imprimindo no arquivo uma label, o tipo de nó e os parâmetros

I/O

Input

A importação não foi feita de forma recursiva, para garantir consistência e poder garantir a validade da árvore, eu usei o fato de que todo nó tem uma label para construir a árvore aos poucos baseado nos nós criados até então utilizando um vetor de nós temporários:

- Se o tipo de nó for uma primitiva: Criamos um nó do respectivo tipo, setamos seus parâmetros utilizando as funções CreatePrimitiveNode e CreateTransformNode mostradas anteriormente e inserimos o Nó retornado no vetor temporário;
- Se o tipo de nó for uma operação booleana: Buscamos no vetor temporário os dois nós com as labels indicadas pelos parâmetros:
    - Se algum nó não for encontrado, retorne erro;
    - Se ambos os nós existirem no vetor, remova-os e crie um novo nó booleano utilizando a CreateBooleanNode para o respectivo tipo de operação utilizando os dois nós retornados como parâmetros da operação booleana e insira o nó booleano de volta no vetor temporário;

I/O

Input

- Se ao fim do arquivo, existir mais de um nó no vetor temporário, o arquivo é inconsistente e retorne erro;
- Se não, o único elemento remanescente no vetor temporário é a nossa raiz alvo, então criamos uma árvore CSG e setamos esse nó como sua raiz, por fim, retornamos a árvore.

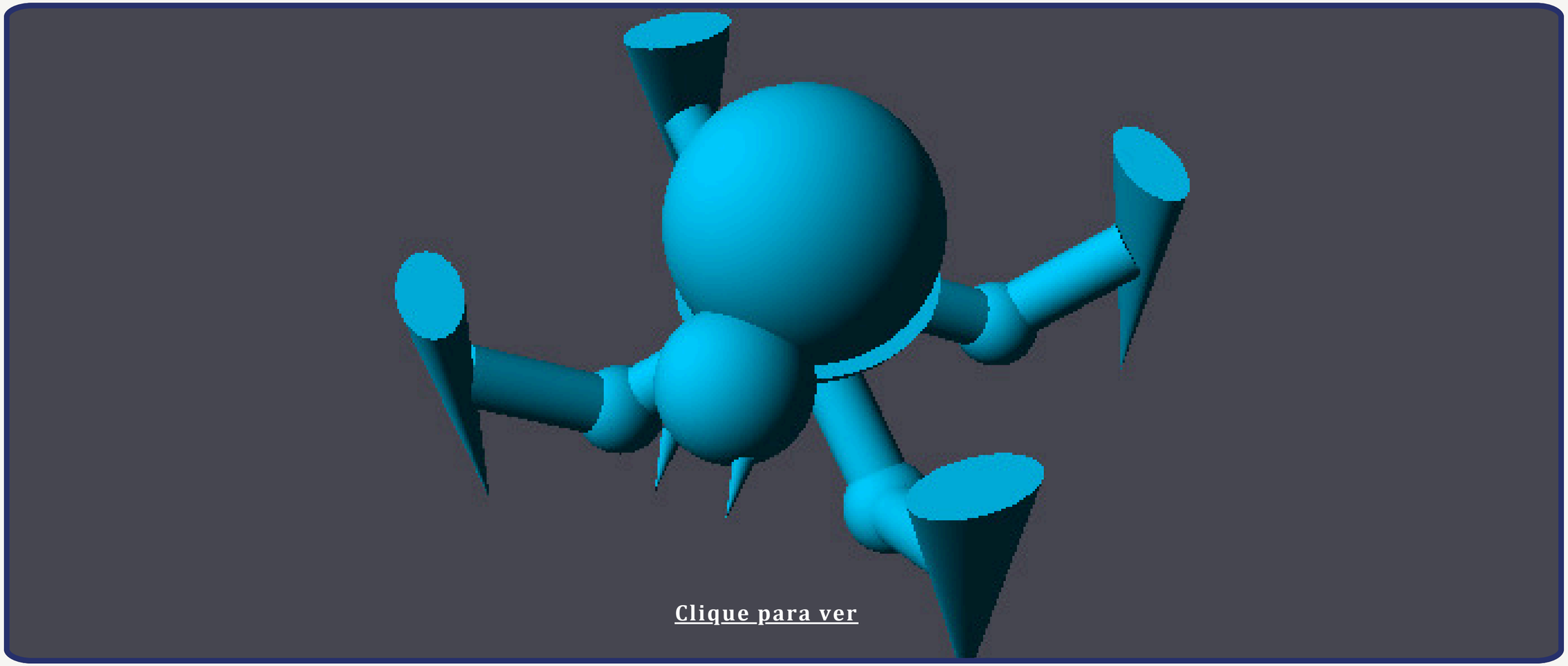
> **OBSERVAÇÃO**
>
> Essa abordagem permite que eu consiga identificar qual linha possui uma label não encontrada ou qual linha gerou erro na árvore diretamente na importação.

PARTE 06

Tema

PARTE 06

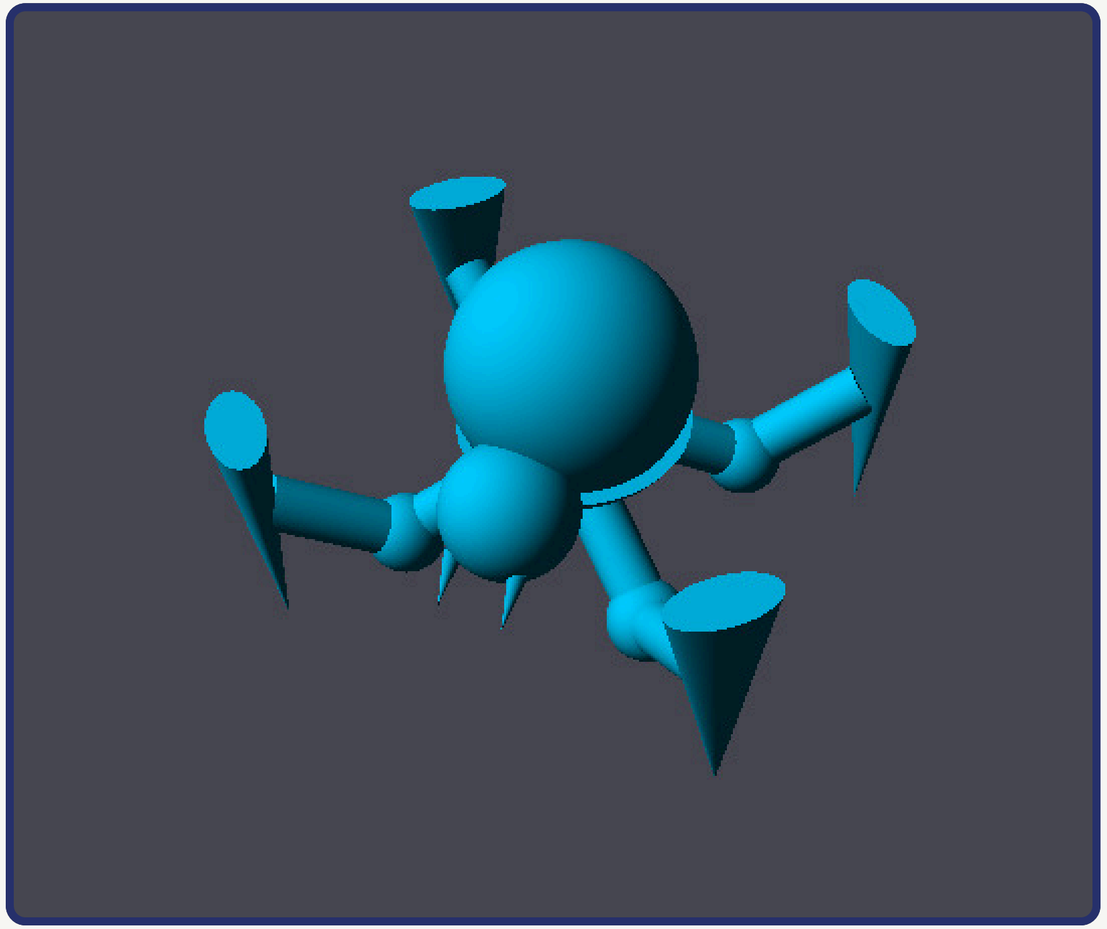
Tema

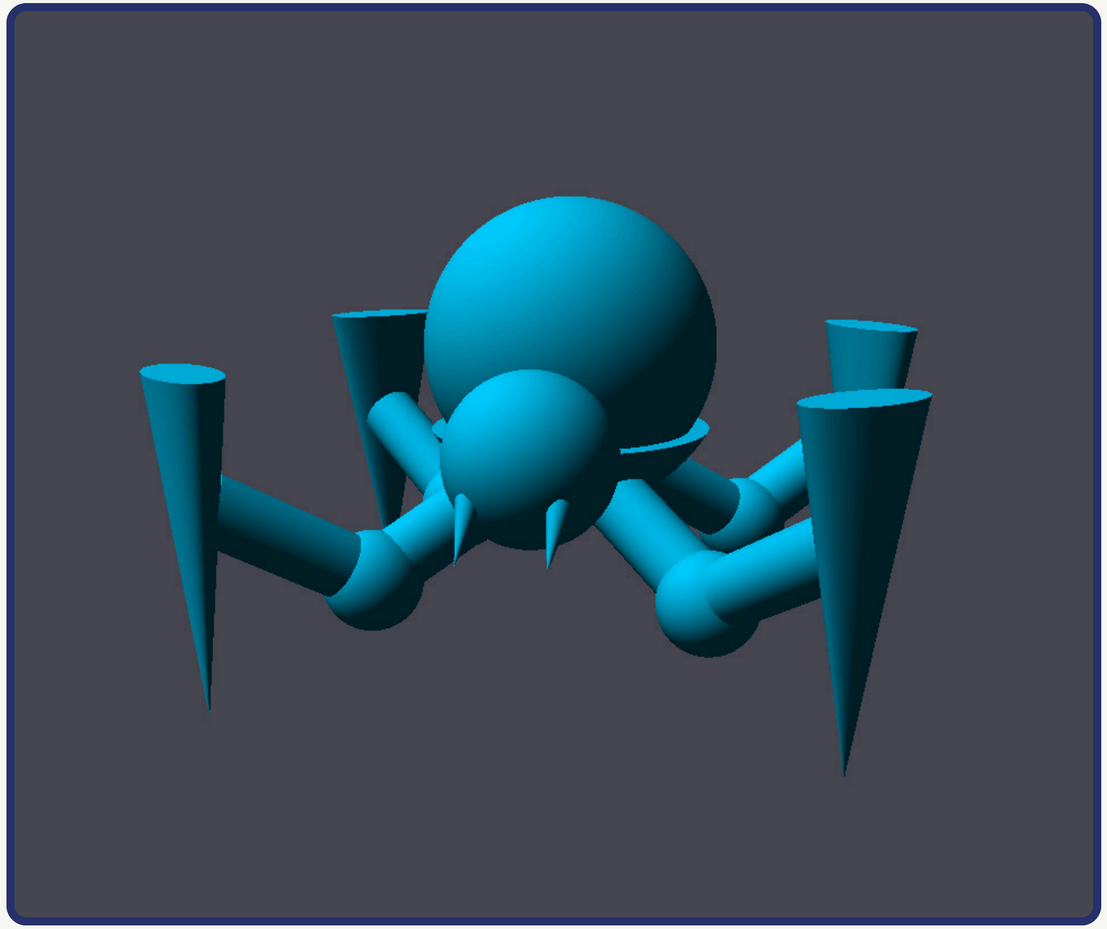


Clique para ver

PARTE 06

Tema







Obrigado!

icaro.mayan@alu.ufc.br

